Comparable And Comparator Interface

23 April 2025 23:07

Comparable Interface: It is present in java.lang package and it contains only one method → compareTo().

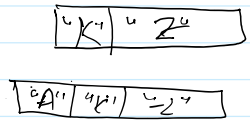
```
public int compareTo(Object obj2)
        obj1.compareTo(obj2);
```

- → returns −ve iff obj1 has to come before obj2.
- → returns +ve iff obj1 has to come after obj2.
- → returns 0 iff obj1 and obj2 are equal.

obj1 = 235, obj2 = 430 235 − 430 = −

```
obj1.compareTo(obj2)
obj2.comparTo(obj)
```

```
TreeSet t = new TreeSet();
t.add('K');
t.add("Z");
t.add("A");
```



t.add("Z"); ⇒ "Z".compareTo("K") +ve.

t.add("A"); ⇒ "A".compareTo("Z") −ve

→ A.compareTo("K") −ve

If we are depending on DNSO then while adding objects into the TreeSet, JVM will call compareTo() method.

Notes:- If DNSO is not available or if we are not satisfied with DNSO then we can go for customized sorting by using Comparator(I).

Comparable(I) is meant for DNSO, whereas Comparator(I) is meant for customized sorting order.

Comparator(I).

Present in java.util package. And, it defines two abstract methods.

① compare() ② equals().

1. public int compare(Object ob1, Object ob2)
   - → returns -ve iff obj1 has to come before obj2.
   - → returns +ve iff obj1 has to come after obj2.
   - → returns 0 iff obj1 and obj2 are equal.

2. public boolean equals(Object ob) :-

Whenever we are implementing Comparator(I) compulsory we should provide only compare() method and we are not required to provide equals() method because it is already available to our class from Object class through inheritance.

```
public class MyComparator implements Comparator {
    @Override
    public int compare(Object ob1, Object ob2) {
        --
        --
    }
    ....
}
```

```java
import java.util.TreeSet;
public class IntegerTreeSet {
    public static void main(String[] args) {
        TreeSet<Integer> t = new TreeSet<Integer>();
        t.add(19);
        t.add(-1);
        t.add(10);
        t.add(7);
        t.add(71);
        t.add(45);
        System.out.println(t);
    }
}
```

Output:

```
[-1, 7, 10, 19, 45, 71]
```

```java
import java.util.Comparator;
public class MyComparator implements Comparator {
    @Override
    public int compare(Object ob1, Object ob2){
        Integer i1 = (Integer) ob1;
        Integer i2 = (Integer) ob2;
        if(i1 < i2){
            return 1;
        }
        else if(i1 > i2){
            return -1;
        }
        else{
            return 0;
        }
    }
}
```

```java
import java.util.TreeSet;
public class IntegerTreeSet {
    public static void main(String[] args) {
        // TreeSet<Integer> t = new TreeSet<Integer>();
        TreeSet t = new TreeSet(new MyComparator());
        t.add(19);
        t.add(-1);
        t.add(10);
        t.add(7);
        t.add(71);
        t.add(45);
        System.out.println(t);
    }
}
```

Output:

```
[71, 45, 19, 10, 7, -1]
```

Comparable vs Comparator.

1. For predefined DNSO we have to use Comparable interface. If we are not satisfied with that DNSO, then we can define our own sorting order by using Comparator interface.

2. For pre-defined non-comparable classes DNSO not already available. so we can defined our own sorting by using Comparator interface.

3. For our own classes like Student, the person who is writing the class is responsible to define DNSO by implementing Comparable(I).

Q. For our own classes are [illegible] the person who is [illegible] the class is responsible to define DNSO by implementing Comparable(I).. The person, who is using our class, if he is not satisfied with DNSO then he can define his own sorting by using Comparator(I).

```
public class Student implements Comparable {
    private int roll;
    private String name;

    public Student(int roll, String name) {
        this.roll = roll;
        this.name = name;
    }

    public int getRoll() {
        return roll;
    }

    public void setRoll(int roll) {
        this.roll = roll;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public int compareTo(Object o){
        Student st = (Student)o;
        if(this.roll > st.roll){
            return 1;
        }
        else if(this.roll < st.roll){
            return -1;
        }
        else{
            return 0;
        }
    }
    @Override
    public String toString() {
        return "Student [roll=" + roll + ", name=" + name + "]";
    }
}
```

```
import java.util.Comparator;
public class MyComparatorStudent implements Comparator {
    @Override
    public int compare(Object ob1, Object ob2){
        Student st1 = (Student)ob1;
        Student st2 = (Student)ob2;
        return st1.getName().compareTo(st2.getName());
    }
}
```

```
import java.util.TreeSet;
public class UEMStudents {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Amit");
        Student s2 = new Student(107, "Harsh");
        Student s3 = new Student(103, "Shiv");
        Student s4 = new Student(106, "Shubh");
        Student s5 = new Student(102, "Rishi");
        TreeSet t = new TreeSet<>();
```

```
        t.add(s1);
        t.add(s2);
        t.add(s3);
        t.add(s4);
        t.add(s5);
        System.out.println(t);
        TreeSet t1 = new TreeSet<>(new MyComparatorStudent());
        t1.add(s1);
        t1.add(s2);
        t1.add(s3);
        t1.add(s4);
        t1.add(s5);
        System.out.println(t1);
    }
}
```

Output:

[Student [roll=101, name=Amit], Student [roll=102, name=Rishi], Student [roll=103, name=Shiv], Student [roll=106, name=Shubh], Student [roll=107, name=Harsh]]

[Student [roll=101, name=Amit], Student [roll=107, name=Harsh], Student [roll=102, name=Rishi], Student [roll=103, name=Shiv], Student [roll=106, name=Shubh]]